

Chuleta + Actividades Resueltas Bases de Datos

UT5.1 Edición de datos · UT5.2 Transacciones · UT5.3 Bloqueos · UT6.1
Programación

PDF de estudio con teoría rápida al principio, ejercicios resueltos, explicación y código SQL comentado listo para repasar antes del examen.

1º DAM - MySQL

Chuleta rápida de examen

Idea clave

Antes de escribir una consulta, piensa en este orden: qué tabla necesito, qué filas quiero, si necesito agrupar, y si voy a modificar datos o solo consultarlos.

1. INSERT, UPDATE y DELETE

```
-- INSERT normal
INSERT INTO tabla (col1, col2) VALUES (valor1, valor2);

-- INSERT usando SELECT: las columnas del INSERT y del SELECT deben cuadrar
INSERT INTO tabla_destino (col1, col2)
SELECT col1, col2
FROM tabla_origen
WHERE condicion;

-- UPDATE: si no pones WHERE, actualizas toda la tabla
UPDATE tabla
SET columna = nuevo_valor
WHERE condicion;

-- DELETE: si no pones WHERE, borras todos los registros
DELETE FROM tabla
WHERE condicion;
```

2. Subconsultas y agregaciones

```
-- Total por cliente
SELECT id_cliente, SUM(total) AS total_compras
FROM pedidos
GROUP BY id_cliente;

-- Filtrar grupos: HAVING se usa despues de GROUP BY
SELECT id_cliente, COUNT(*) AS total_pedidos
FROM pedidos
GROUP BY id_cliente
HAVING COUNT(*) > 2;

-- Comparar contra una media general
WHERE total_compras > (SELECT AVG(total_compras) FROM (... ) x);
```

3. Transacciones

```
START TRANSACTION;
-- operaciones relacionadas
COMMIT; -- confirma todo
ROLLBACK; -- deshace todo si algo falla
```

4. Bloqueos

```
START TRANSACTION;
SELECT * FROM tabla WHERE id = 1 FOR UPDATE; -- bloquea esa fila
-- comprobar y modificar
COMMIT;
```

5. Procedimientos y funciones en MySQL

```

DELIMITER $$
DROP PROCEDURE IF EXISTS mi_proc $$
CREATE PROCEDURE mi_proc(IN p_id INT, OUT resultado BOOLEAN)
BEGIN
    SET resultado = TRUE;
END $$
DELIMITER ;
CALL mi_proc(1, @ok);
SELECT @ok;

DELIMITER $$
DROP FUNCTION IF EXISTS mi_func $$
CREATE FUNCTION mi_func(p_num INT) RETURNS INT
DETERMINISTIC
BEGIN
    RETURN p_num * 2;
END $$
DELIMITER ;
SELECT mi_func(5);

```

6. IF, CASE y bucles

```

IF condicion THEN
    SELECT 'ok';
ELSEIF otra_condicion THEN
    SELECT 'otra';
ELSE
    SELECT 'error';
END IF;

CASE valor
    WHEN 1 THEN SELECT 'Lunes';
    WHEN 2 THEN SELECT 'Martes';
    ELSE SELECT 'Error';
END CASE;

WHILE contador <= limite DO
    SET contador = contador + 1;
END WHILE;

```

7. Cursores, errores y triggers

```

-- Cursor: recorrer una consulta fila a fila
DECLARE fin BOOL DEFAULT FALSE;
DECLARE cur CURSOR FOR SELECT id, nombre FROM tabla;
DECLARE CONTINUE HANDLER FOR NOT FOUND SET fin = TRUE;

-- Error personalizado
SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Mensaje de error';

-- Trigger
CREATE TRIGGER nombre_trigger
BEFORE INSERT ON tabla
FOR EACH ROW
BEGIN
    -- OLD.columna en UPDATE/DELETE, NEW.columna en INSERT/UPDATE
END;

```

UT5.1 - Edición de datos: INSERT, UPDATE y DELETE

Actividad 1 - Clientes y pedidos

Idea clave

El error típico aquí es usar p.total_compras. Esa columna no existe en pedidos; hay que calcularla con SUM(total) y GROUP BY.

```

-- 1) Crear tabla e insertar clientes cuyo total de compras supera la media
CREATE TABLE clientes_premium (
  id_cliente INT,
  nombre VARCHAR(50),
  total_compras DECIMAL(10,2)
);

INSERT INTO clientes_premium (id_cliente, nombre, total_compras)
SELECT c.id_cliente, c.nombre, SUM(p.total) AS total_compras
FROM clientes c
INNER JOIN pedidos p ON p.id_cliente = c.id_cliente
GROUP BY c.id_cliente, c.nombre
HAVING SUM(p.total) > (
  SELECT AVG(t.total_compras)
  FROM (
    SELECT SUM(total) AS total_compras
    FROM pedidos
    GROUP BY id_cliente
  ) t
);

-- 2) Poner inactivos los clientes con compras menores a la media general
UPDATE clientes c
SET activo = FALSE
WHERE c.id_cliente IN (
  SELECT x.id_cliente
  FROM (
    SELECT p.id_cliente, SUM(p.total) AS total_compras
    FROM pedidos p
    GROUP BY p.id_cliente
  ) x
  WHERE x.total_compras < (
    SELECT AVG(y.total_compras)
    FROM (
      SELECT SUM(total) AS total_compras
      FROM pedidos
      GROUP BY id_cliente
    ) y
  )
);

-- 3) Eliminar pedidos de clientes inactivos
DELETE FROM pedidos
WHERE id_cliente IN (
  SELECT id_cliente FROM clientes WHERE activo = FALSE
);

-- 4) Crear una tabla similar a clientes e insertar clientes sin pedidos
CREATE TABLE clientes_sin_pedidos AS
SELECT *
FROM clientes
WHERE id_cliente NOT IN (
  SELECT DISTINCT id_cliente FROM pedidos
);

-- 5) Aumentar un 10% el total de pedidos de clientes de Madrid
UPDATE pedidos p
INNER JOIN clientes c ON c.id_cliente = p.id_cliente
SET p.total = p.total * 1.10
WHERE c.ciudad = 'Madrid';

-- 6) Eliminar clientes que nunca hayan hecho un pedido
DELETE FROM clientes
WHERE id_cliente NOT IN (
  SELECT DISTINCT id_cliente FROM pedidos
);

```

Resultado esperado

Ana tiene 700 euros en pedidos, Luis 150, Carlos 50 y Sofia no tiene pedidos. Por eso la media por cliente con pedidos es 300; Ana entra en clientes_premium.

Actividad 2 - Usuarios y préstamos

```
-- 1) Usuarios con mas prestamos que el promedio por usuario
CREATE TABLE usuarios_frecuentes (
  id_usuario INT,
  nombre VARCHAR(50),
  total_prestamos INT
);

INSERT INTO usuarios_frecuentes (id_usuario, nombre, total_prestamos)
SELECT u.id_usuario, u.nombre, COUNT(p.id_prestamo) AS total_prestamos
FROM usuarios u
INNER JOIN prestamos p ON p.id_usuario = u.id_usuario
GROUP BY u.id_usuario, u.nombre
HAVING COUNT(p.id_prestamo) > (
  SELECT AVG(t.total_prestamos)
  FROM (
    SELECT COUNT(*) AS total_prestamos
    FROM prestamos
    GROUP BY id_usuario
  ) t
);

-- 2) Inactivar usuarios que no tengan ningun prestamo activo
-- Prestamo activo = fecha_devolucion IS NULL
UPDATE usuarios u
SET activo = FALSE
WHERE u.id_usuario NOT IN (
  SELECT DISTINCT id_usuario
  FROM prestamos
  WHERE fecha_devolucion IS NULL
);

-- 3) Eliminar prestamos de usuarios inactivos
DELETE FROM prestamos
WHERE id_usuario IN (
  SELECT id_usuario FROM usuarios WHERE activo = FALSE
);

-- 4) Crear tabla similar a usuarios con usuarios que no hayan hecho prestamos
CREATE TABLE usuarios_sin_prestamos AS
SELECT *
FROM usuarios
WHERE id_usuario NOT IN (
  SELECT DISTINCT id_usuario FROM prestamos
);

-- 5) Eliminar usuarios que nunca hayan hecho un prestamo
DELETE FROM usuarios
WHERE id_usuario NOT IN (
  SELECT DISTINCT id_usuario FROM prestamos
);
```

Idea clave

Para saber quién no tiene préstamos activos se usa NOT IN sobre los usuarios que sí tienen algún fecha_devolucion IS NULL.

UT5.2 - Transacciones

Actividad 1 - Compra con stock y rollback

Idea clave

La idea es proteger la operación completa: comprobar stock, descontar stock, crear pedido y confirmar. Si algo falla, ROLLBACK.

```
-- Ejemplo de solución general para venta de producto
START TRANSACTION;

-- Bloqueamos el producto para que nadie modifique el stock mientras compramos
SELECT stock
FROM productos
WHERE id = 1
FOR UPDATE;

-- Si hay stock suficiente, se descuenta
UPDATE productos
SET stock = stock - 1
WHERE id = 1 AND stock >= 1;

-- Aquí irían el INSERT del pedido y del detalle del pedido
-- INSERT INTO pedidos (...);
-- INSERT INTO detalle_pedidos (...);

-- Si todo ha ido bien:
COMMIT;

-- Si hubiera cualquier error antes del COMMIT:
-- ROLLBACK;
```

Cuidado

No sirve hacer solo UPDATE productos SET stock = stock - 1. Si dos usuarios compran a la vez, puedes vender más unidades de las disponibles. Por eso se bloquea la fila con FOR UPDATE.

UT5.3 - Bloqueos

Actividad 1 - Luisa paga 80 y envía 40 a Miguel

Idea clave

Luisa tiene 100 euros. Si dos operaciones se ejecutan a la vez, $80 + 40 = 120$ y no debería permitirse. La solución es bloquear la fila de Luisa antes de comprobar el saldo.

```
-- SESION 1: Luisa paga una cena de 80 euros
START TRANSACTION;

SELECT saldo
FROM cuentas
WHERE id_cuenta = 1
FOR UPDATE;

-- Como 100 >= 80, esta operacion puede hacerse
UPDATE cuentas
SET saldo = saldo - 80
WHERE id_cuenta = 1 AND saldo >= 80;

COMMIT;

-- SESION 2: Luisa intenta enviar 40 euros a Miguel
START TRANSACTION;

SELECT saldo
FROM cuentas
WHERE id_cuenta = 1
FOR UPDATE;

-- Esta sesion esperara hasta que la sesion 1 haga COMMIT.
-- Despues Luisa tendra 20, por tanto no se cumple saldo >= 40.
UPDATE cuentas
SET saldo = saldo - 40
WHERE id_cuenta = 1 AND saldo >= 40;

-- Solo si el UPDATE ha afectado 1 fila, se suma a Miguel.
UPDATE cuentas
SET saldo = saldo + 40
WHERE id_cuenta = 2;

-- Si no se pudo restar a Luisa, se debe deshacer:
ROLLBACK;
-- Si si se pudo restar a Luisa y sumar a Miguel:
-- COMMIT;
```

Resultado esperado

Con FOR UPDATE, la segunda sesión no puede leer y modificar el saldo antiguo de Luisa hasta que la primera termine. Así se evita gastar 120 teniendo solo 100.

UT6.1 - Programación de bases de datos

Actividad 1 - Procedimiento que muestra la versión de MySQL

```
DELIMITER $$
DROP PROCEDURE IF EXISTS mostrarVersion $$
CREATE PROCEDURE mostrarVersion()
BEGIN
    SELECT VERSION() AS version_mysql;
END $$
DELIMITER ;
CALL mostrarVersion();
```

Actividad 2 - Procedimiento Hola + nombre

```
DELIMITER $$
DROP PROCEDURE IF EXISTS saludarNombre $$
CREATE PROCEDURE saludarNombre(IN p_nombre VARCHAR(50))
BEGIN
    SELECT CONCAT('Hola ', p_nombre) AS saludo;
END $$
DELIMITER ;
CALL saludarNombre('Ryad');
```

Actividad 3 - Función Hola + nombre

```
DELIMITER $$
DROP FUNCTION IF EXISTS f_saludarNombre $$
CREATE FUNCTION f_saludarNombre(p_nombre VARCHAR(50))
RETURNS VARCHAR(100)
DETERMINISTIC
BEGIN
    RETURN CONCAT('Hola ', p_nombre);
END $$
DELIMITER ;
SELECT f_saludarNombre('Ana');
```

Actividad 4 - Función que concatena dos argumentos en mayúsculas

```
DELIMITER $$
DROP FUNCTION IF EXISTS concatMayus $$
CREATE FUNCTION concatMayus(arg1 VARCHAR(50), arg2 VARCHAR(50))
RETURNS VARCHAR(101)
DETERMINISTIC
BEGIN
    RETURN CONCAT(UPPER(arg1), '-', UPPER(arg2));
END $$
DELIMITER ;
SELECT concatMayus('hola', 'mundo'); -- HOLA-MUNDO
```

Actividad 5 - Mostrar pagos iguales o mayores a una cantidad

```
DELIMITER $$
DROP PROCEDURE IF EXISTS pagosMinimos $$
CREATE PROCEDURE pagosMinimos(IN p_cantidad NUMERIC(5,2))
BEGIN
    SELECT nombre, cantidad
    FROM pagos
    WHERE cantidad >= p_cantidad
    ORDER BY cantidad DESC;
END $$
DELIMITER ;
CALL pagosMinimos(300);
```

Actividad 6 - Insertar un pago

```

DELIMITER $$
DROP PROCEDURE IF EXISTS insertarPago $$
CREATE PROCEDURE insertarPago(IN p_nombre VARCHAR(100), IN p_cantidad NUMERIC(5,2))
BEGIN
    INSERT INTO pagos (nombre, cantidad)
    VALUES (p_nombre, p_cantidad);
END $$
DELIMITER ;
CALL insertarPago('Paco', 250);

```

Actividad 7 - Mostrar el mayor de dos números

```

DELIMITER $$
DROP PROCEDURE IF EXISTS mostrarMayor $$
CREATE PROCEDURE mostrarMayor(IN n1 INT, IN n2 INT)
BEGIN
    IF n1 > n2 THEN
        SELECT n1 AS mayor;
    ELSEIF n2 > n1 THEN
        SELECT n2 AS mayor;
    ELSE
        SELECT 'Son iguales' AS resultado;
    END IF;
END $$
DELIMITER ;
CALL mostrarMayor(8, 5);

```

Actividad 8 - Nota entre 1 y 10

```

DELIMITER $$
DROP PROCEDURE IF EXISTS mostrarNota $$
CREATE PROCEDURE mostrarNota(IN nota DECIMAL(4,2))
BEGIN
    IF nota < 1 OR nota > 10 THEN
        SELECT 'Error: la nota debe estar entre 1 y 10' AS resultado;
    ELSEIF nota < 5 THEN
        SELECT 'Insuficiente' AS resultado;
    ELSEIF nota < 6 THEN
        SELECT 'Suficiente' AS resultado;
    ELSEIF nota < 7 THEN
        SELECT 'Bien' AS resultado;
    ELSEIF nota < 9 THEN
        SELECT 'Notable' AS resultado;
    ELSE
        SELECT 'Sobresaliente' AS resultado;
    END IF;
END $$
DELIMITER ;

```

Actividad 9 - Día de la semana

```

DELIMITER $$
DROP PROCEDURE IF EXISTS diaSemana $$
CREATE PROCEDURE diaSemana(IN dia INT)
BEGIN
    CASE dia
        WHEN 1 THEN SELECT 'Lunes' AS dia;
        WHEN 2 THEN SELECT 'Martes' AS dia;
        WHEN 3 THEN SELECT 'Miercoles' AS dia;
        WHEN 4 THEN SELECT 'Jueves' AS dia;
        WHEN 5 THEN SELECT 'Viernes' AS dia;
        WHEN 6 THEN SELECT 'Sabado' AS dia;
        WHEN 7 THEN SELECT 'Domingo' AS dia;
        ELSE SELECT 'Error: numero fuera de rango' AS dia;
    END CASE;
END $$
DELIMITER ;

```

Actividad 10 - Resultado de quiniela


```

DELIMITER $$
DROP PROCEDURE IF EXISTS quiniela $$
CREATE PROCEDURE quiniela(IN goles_local INT, IN goles_visitante INT)
BEGIN
    IF goles_local > goles_visitante THEN
        SELECT '1' AS quiniela; -- gana local
    ELSEIF goles_local = goles_visitante THEN
        SELECT 'X' AS quiniela; -- empate
    ELSE
        SELECT '2' AS quiniela; -- gana visitante
    END IF;
END $$
DELIMITER ;

```

Actividad 11 - Función calculadora

```

DELIMITER $$
DROP FUNCTION IF EXISTS calculadora $$
CREATE FUNCTION calculadora(a DECIMAL(10,2), b DECIMAL(10,2), operacion INT)
RETURNS DECIMAL(10,2)
DETERMINISTIC
BEGIN
    IF operacion = 1 THEN
        RETURN a + b;
    ELSEIF operacion = 2 THEN
        RETURN a - b;
    ELSEIF operacion = 3 THEN
        RETURN a * b;
    ELSEIF operacion = 4 THEN
        IF b = 0 THEN
            RETURN NULL;
        END IF;
        RETURN a / b;
    ELSE
        RETURN NULL;
    END IF;
END $$
DELIMITER ;
SELECT calculadora(10, 2, 4);

```

Actividad 12 - Validar límites y mostrar pares

```

DELIMITER $$
DROP FUNCTION IF EXISTS validarLmites $$
CREATE FUNCTION validarLmites(inferior INT, superior INT)
RETURNS BOOLEAN
DETERMINISTIC
BEGIN
    IF inferior > 0 AND superior > 0 AND inferior < superior THEN
        RETURN TRUE;
    END IF;
    RETURN FALSE;
END $$
DELIMITER ;

DELIMITER $$
DROP PROCEDURE IF EXISTS mostrarPares $$
CREATE PROCEDURE mostrarPares(IN inferior INT, IN superior INT)
BEGIN
    DECLARE i INT;
    DECLARE resultado VARCHAR(500) DEFAULT '';

    IF validarLmites(inferior, superior) THEN
        SET i = inferior;
        WHILE i <= superior DO
            IF MOD(i, 2) = 0 THEN
                SET resultado = CONCAT(resultado, i, ' ');
            END IF;
            SET i = i + 1;
        END WHILE;
        SELECT resultado AS pares;
    ELSE
        SELECT 'Lmites incorrectos' AS error;
    END IF;
END $$
DELIMITER ;

```

Actividad 13 - Sumar impares entre límites

```

DELIMITER $$
DROP PROCEDURE IF EXISTS sumarImpares $$
CREATE PROCEDURE sumarImpares(IN inferior INT, IN superior INT)
BEGIN
    DECLARE i INT;
    DECLARE suma INT DEFAULT 0;

    IF validarLmites(inferior, superior) THEN
        SET i = inferior;
        WHILE i <= superior DO
            IF MOD(i, 2) <> 0 THEN
                SET suma = suma + i;
            END IF;
            SET i = i + 1;
        END WHILE;
        SELECT suma AS suma_impares;
    ELSE
        SELECT 'Lmites incorrectos' AS error;
    END IF;
END $$
DELIMITER ;

```

Actividad 14 - Tablas de multiplicar del 0 al 9

```

DELIMITER $$
DROP PROCEDURE IF EXISTS tablasMultiplicar $$
CREATE PROCEDURE tablasMultiplicar()
BEGIN
    DECLARE tabla INT DEFAULT 0;
    DECLARE n INT;
    DECLARE linea VARCHAR(200);

    WHILE tabla <= 9 DO
        SET n = 0;
        SET linea = CONCAT('Tabla del ', tabla, ': ');
        WHILE n <= 10 DO
            SET linea = CONCAT(linea, tabla, 'x', n, '=', tabla*n, ' | ');
            SET n = n + 1;
        END WHILE;
        SELECT linea AS tabla_multiplicar;
        SET tabla = tabla + 1;
    END WHILE;
END $$
DELIMITER ;

```

Actividad 15 - Función número primo

```

DELIMITER $$
DROP FUNCTION IF EXISTS esPrimo $$
CREATE FUNCTION esPrimo(n INT)
RETURNS BOOLEAN
DETERMINISTIC
BEGIN
    DECLARE divisor INT DEFAULT 2;

    IF n < 2 THEN
        RETURN FALSE;
    END IF;

    WHILE divisor * divisor <= n DO
        IF MOD(n, divisor) = 0 THEN
            RETURN FALSE;
        END IF;
        SET divisor = divisor + 1;
    END WHILE;

    RETURN TRUE;
END $$
DELIMITER ;

```

Actividad 16 - Insertar primos entre dos límites

```

CREATE TABLE IF NOT EXISTS numeros_primos (
  numero NUMERIC(5,2) NOT NULL PRIMARY KEY
);

DELIMITER $$
DROP PROCEDURE IF EXISTS guardarPrimos $$
CREATE PROCEDURE guardarPrimos(IN inferior INT, IN superior INT)
BEGIN
  DECLARE i INT;

  IF validarLmites(inferior, superior) THEN
    DELETE FROM numeros_primos;
    SET i = inferior;
    WHILE i <= superior DO
      IF esPrimo(i) THEN
        INSERT INTO numeros_primos (numero) VALUES (i);
      END IF;
      SET i = i + 1;
    END WHILE;
    SELECT * FROM numeros_primos;
  ELSE
    SELECT 'Lmites incorrectos' AS error;
  END IF;
END $$
DELIMITER ;

```

Actividad 17 - Cursor: subir salarios por departamento

```

DELIMITER $$
DROP PROCEDURE IF EXISTS actualizarSalarios $$
CREATE PROCEDURE actualizarSalarios()
BEGIN
  DECLARE v_id INT;
  DECLARE v_departamento VARCHAR(50);
  DECLARE fin BOOL DEFAULT FALSE;

  DECLARE cur CURSOR FOR
    SELECT id, departamento FROM empleados;
  DECLARE CONTINUE HANDLER FOR NOT FOUND SET fin = TRUE;

  OPEN cur;
  bucle: LOOP
    FETCH cur INTO v_id, v_departamento;
    IF fin THEN
      LEAVE bucle;
    END IF;

    IF v_departamento = 'Ventas' THEN
      UPDATE empleados SET salario = salario * 1.10 WHERE id = v_id;
    ELSEIF v_departamento = 'IT' THEN
      UPDATE empleados SET salario = salario * 1.05 WHERE id = v_id;
    END IF;
  END LOOP;
  CLOSE cur;

  SELECT * FROM empleados;
END $$
DELIMITER ;

```

Actividad 18 - Procedimiento procesar pedido

```

DELIMITER $$
DROP PROCEDURE IF EXISTS procesar_pedido $$
CREATE PROCEDURE procesar_pedido(
    IN p_id_cliente INT,
    IN p_id_producto INT,
    IN p_cantidad INT,
    OUT resultadoOK BOOLEAN,
    OUT mensaje_error VARCHAR(100)
)
BEGIN
    DECLARE v_stock INT;
    DECLARE v_precio NUMERIC(5,2);
    DECLARE v_saldo NUMERIC(10,2);
    DECLARE v_total NUMERIC(10,2);
    DECLARE v_id_pedido INT;

    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        SET resultadoOK = FALSE;
        SET mensaje_error = 'Error inesperado. Operacion cancelada';
    END;

    START TRANSACTION;

    SELECT saldo INTO v_saldo
    FROM clientes
    WHERE id_cliente = p_id_cliente
    FOR UPDATE;

    SELECT stock, precio INTO v_stock, v_precio
    FROM productos
    WHERE id_producto = p_id_producto
    FOR UPDATE;

    SET v_total = v_precio * p_cantidad;

    IF p_cantidad <= 0 THEN
        SET resultadoOK = FALSE;
        SET mensaje_error = 'La cantidad debe ser positiva';
        ROLLBACK;
    ELSEIF v_stock < p_cantidad THEN
        SET resultadoOK = FALSE;
        SET mensaje_error = 'Stock insuficiente';
        ROLLBACK;
    ELSEIF v_saldo < v_total THEN
        SET resultadoOK = FALSE;
        SET mensaje_error = 'Saldo insuficiente';
        ROLLBACK;
    ELSE
        INSERT INTO pedidos (id_cliente, fecha_hora)
        VALUES (p_id_cliente, NOW());
        SET v_id_pedido = LAST_INSERT_ID();

        INSERT INTO detalle_pedidos (id_pedido, id_producto, cantidad)
        VALUES (v_id_pedido, p_id_producto, p_cantidad);

        UPDATE productos
        SET stock = stock - p_cantidad
        WHERE id_producto = p_id_producto;

        UPDATE clientes
        SET saldo = saldo - v_total
        WHERE id_cliente = p_id_cliente;

        SET resultadoOK = TRUE;
        SET mensaje_error = 'Pedido procesado correctamente';
        COMMIT;
    END IF;
END $$
DELIMITER ;

```

Idea clave

El FOR UPDATE bloquea cliente y producto. Así no se vende el mismo stock dos veces ni se gasta el mismo saldo dos veces.

Actividad 19 - Procedimiento gestionar reserva

```

DELIMITER $$
DROP PROCEDURE IF EXISTS gestionar_reserva $$
CREATE PROCEDURE gestionar_reserva(
    IN p_id_cliente INT,
    IN p_numero_habitacion NUMERIC(3,0),
    IN p_fecha_inicio DATE,
    IN p_fecha_fin DATE,
    OUT resultadoOK BOOLEAN,
    OUT mensaje_error VARCHAR(100)
)
BEGIN
    DECLARE v_saldo NUMERIC(10,2);
    DECLARE v_precio NUMERIC(6,2);
    DECLARE v_noches INT;
    DECLARE v_total NUMERIC(10,2);

    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        SET resultadoOK = FALSE;
        SET mensaje_error = 'Error inesperado. Reserva cancelada';
    END;

    START TRANSACTION;

    IF p_fecha_inicio IS NULL OR p_fecha_fin IS NULL OR p_fecha_inicio >= p_fecha_fin THEN
        SET resultadoOK = FALSE;
        SET mensaje_error = 'Fechas incorrectas';
        ROLLBACK;
    ELSE
        SELECT saldo INTO v_saldo
        FROM clientes
        WHERE id_cliente = p_id_cliente
        FOR UPDATE;

        SELECT precio_noche INTO v_precio
        FROM habitaciones
        WHERE numero_habitacion = p_numero_habitacion
        FOR UPDATE;

        SET v_noches = DATEDIFF(p_fecha_fin, p_fecha_inicio);
        SET v_total = v_noches * v_precio;

        IF v_saldo < v_total THEN
            SET resultadoOK = FALSE;
            SET mensaje_error = 'Saldo insuficiente';
            ROLLBACK;
        ELSE
            INSERT INTO reservas (id_cliente, numero_habitacion, fecha_entrada, fecha_salida)
            VALUES (p_id_cliente, p_numero_habitacion, p_fecha_inicio, p_fecha_fin);

            UPDATE clientes
            SET saldo = saldo - v_total
            WHERE id_cliente = p_id_cliente;

            SET resultadoOK = TRUE;
            SET mensaje_error = 'Reserva realizada correctamente';
            COMMIT;
        END IF;
    END IF;
END $$
DELIMITER ;

```

Actividad 20 - Triggers de auditoría de salarios

```

DELIMITER $$
DROP TRIGGER IF EXISTS trg_empleados_bi $$
CREATE TRIGGER trg_empleados_bi
BEFORE INSERT ON empleados
FOR EACH ROW
BEGIN
    IF NEW.salario <= 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'El salario debe ser positivo';
    END IF;
END $$

DROP TRIGGER IF EXISTS trg_empleados_bu $$
CREATE TRIGGER trg_empleados_bu
BEFORE UPDATE ON empleados
FOR EACH ROW
BEGIN
    IF NEW.salario <= 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'El salario debe ser positivo';
    END IF;

    IF NEW.salario <> OLD.salario THEN
        INSERT INTO empleados_auditoria (empleado_id, salario_antiguo, salario_nuevo)
        VALUES (OLD.id, OLD.salario, NEW.salario);
    END IF;
END $$
DELIMITER ;

```

Cuidado

En un BEFORE INSERT aún no existe salario antiguo, por eso solo se valida. En BEFORE UPDATE sí existen OLD.salario y NEW.salario.

Actividad 21 - Triggers para evitar solape de reservas

```

DELIMITER $$
DROP TRIGGER IF EXISTS trg_reservas_bi $$
CREATE TRIGGER trg_reservas_bi
BEFORE INSERT ON reservas
FOR EACH ROW
BEGIN
    IF NEW.fecha_entrada >= NEW.fecha_salida THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Fechas incorrectas';
    END IF;

    IF EXISTS (
        SELECT 1
        FROM reservas r
        WHERE r.numero_habitacion = NEW.numero_habitacion
        -- Hay solape si la nueva entrada es antes de la salida existente
        -- y la nueva salida es despues de la entrada existente.
        AND NEW.fecha_entrada < r.fecha_salida
        AND NEW.fecha_salida > r.fecha_entrada
    ) THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'La habitacion ya esta reservada en esas fechas';
    END IF;
END $$

DROP TRIGGER IF EXISTS trg_reservas_bu $$
CREATE TRIGGER trg_reservas_bu
BEFORE UPDATE ON reservas
FOR EACH ROW
BEGIN
    IF NEW.fecha_entrada >= NEW.fecha_salida THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Fechas incorrectas';
    END IF;

    IF EXISTS (
        SELECT 1
        FROM reservas r
        WHERE r.numero_habitacion = NEW.numero_habitacion
        AND r.id_reserva <> OLD.id_reserva
        AND NEW.fecha_entrada < r.fecha_salida
        AND NEW.fecha_salida > r.fecha_entrada
    ) THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'La habitacion ya esta reservada en esas fechas';
    END IF;
END $$
DELIMITER ;

```

Idea clave

La fórmula del solape es: $\text{nueva_entrada} < \text{salida_existente}$ AND $\text{nueva_salida} > \text{entrada_existente}$.
Permite que una reserva empiece justo el día de salida de otra.

Errores típicos que debes evitar